



Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores

CE1107: Fundamentos de Arquitectura de Computadores

Bitácora del Proyecto

Cerrojo

Estudiantes:

Guillermo Sánchez Cedeño
Kendall Fernández Obando

Profesor: Luis Chavarría Zamora

Semestre: II Semestre 2026

29 de septiembre de 2026

Índice

1	7 de septiembre de 2026	2
1.1	Codificador de señas	2
1.2	Tabla de verdad	2
1.3	Mapas de Karnaugh	2
1.4	Ecuaciones obtenidas	3
1.5	Simulación en Proteus	4
2	11 de septiembre de 2026	6
2.1	Verificador de señas	6
2.2	Tabla de verdad y ecuación del verificador	6
2.3	Implementación del verificador	8
2.4	Herramientas de apoyo	8
2.5	Bloque de reloj	9
2.6	Antirrebote del botón	9
2.7	Oscilador con el NE555	9
2.8	Contador de pulsos y señal DONE	10
2.9	Serializador	11
2.10	Filtrado del reloj	12
2.11	Cadena de registros de desplazamiento	12
2.12	Orden de los bits en el registro	13
2.13	Comparador de contraseñas	14
3	13 de septiembre de 2026	16
3.1	Controlador de estados	16
3.2	Contador de dígitos	16
3.3	Cerrojo de posición de la puerta	16
3.4	Monoestable de reinicio	17
3.5	Ecuaciones de estado	17
3.6	Salidas hacia el motor	17
3.7	Decodificador de anuncios	18
3.8	Simulación	20
4	Repositorio de GitHub	21

1 7 de septiembre de 2026

1.1 Codificador de señas

Para esta sesión trabajamos en el codificador, que es el bloque encargado de tomar la lectura de los sensores de la mano y comprimirla a un número binario de tres bits que el resto del circuito pueda manejar con comodidad. La idea del proyecto es que el usuario ingrese cada dígito de la contraseña levantando una cantidad determinada de dedos, entonces necesitamos alguna manera de traducir esa seña a un valor numérico antes de poder guardarla o compararla.

El sistema cuenta con cuatro sensores, uno por dedo, a los que llamamos S_1 , S_2 , S_3 y S_4 . La seña válida es la que levanta los dedos en orden, es decir que para representar el número tres se activan S_1 , S_2 y S_3 . Ahora bien, nada le impide físicamente al usuario levantar una combinación que no siga ese orden, como por ejemplo levantar únicamente el dedo del medio, pero de esos casos no se encarga este bloque.

Para eso tenemos un verificador aparte, cuya única responsabilidad es revisar si la seña que se está ingresando respeta el orden y avisarle al resto del sistema cuando no lo hace. Esta división de responsabilidades es la que nos permite diseñar el codificador asumiendo que la seña que le llega ya viene bien formada, sin tener que meterle a este bloque la lógica de validación encima.

Como tenemos cuatro sensores, en total existen 16 combinaciones posibles de entrada, de las cuales únicamente 5 corresponden a señas válidas, que son las de los valores del 0 al 4. Las otras 11 son las que el verificador se encarga de rechazar.

1.2 Tabla de verdad

Empezamos construyendo la tabla de verdad con las cinco combinaciones válidas, relacionando el estado de los sensores con la cantidad de dedos levantados y con el valor binario de tres bits que le corresponde en la salida.

S_1	S_2	S_3	S_4	Dedos	B_2	B_1	B_0
0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	1
1	1	0	0	2	0	1	0
1	1	1	0	3	0	1	1
1	1	1	1	4	1	0	0

Tabla 1: Tabla de verdad del codificador de señas.

1.3 Mapas de Karnaugh

Con la tabla lista pasamos a los mapas de Karnaugh, uno por cada bit de salida. Las 11 combinaciones que no respetan el orden de los dedos las marcamos como condiciones libres. Esto no significa que no puedan ocurrir, sino que cuando ocurren es el verificador el que las

detecta y anuncia que la seña es inválida, entonces la salida que produzca el codificador en esos casos no se va a utilizar para nada y nos da lo mismo cuál sea. Aprovechar esa libertad resultó ser una ventaja enorme, porque nos permitió estirar los grupos hasta cubrir esas casillas y llegar a ecuaciones mucho más cortas de las que hubiéramos obtenido tratándolas como ceros.

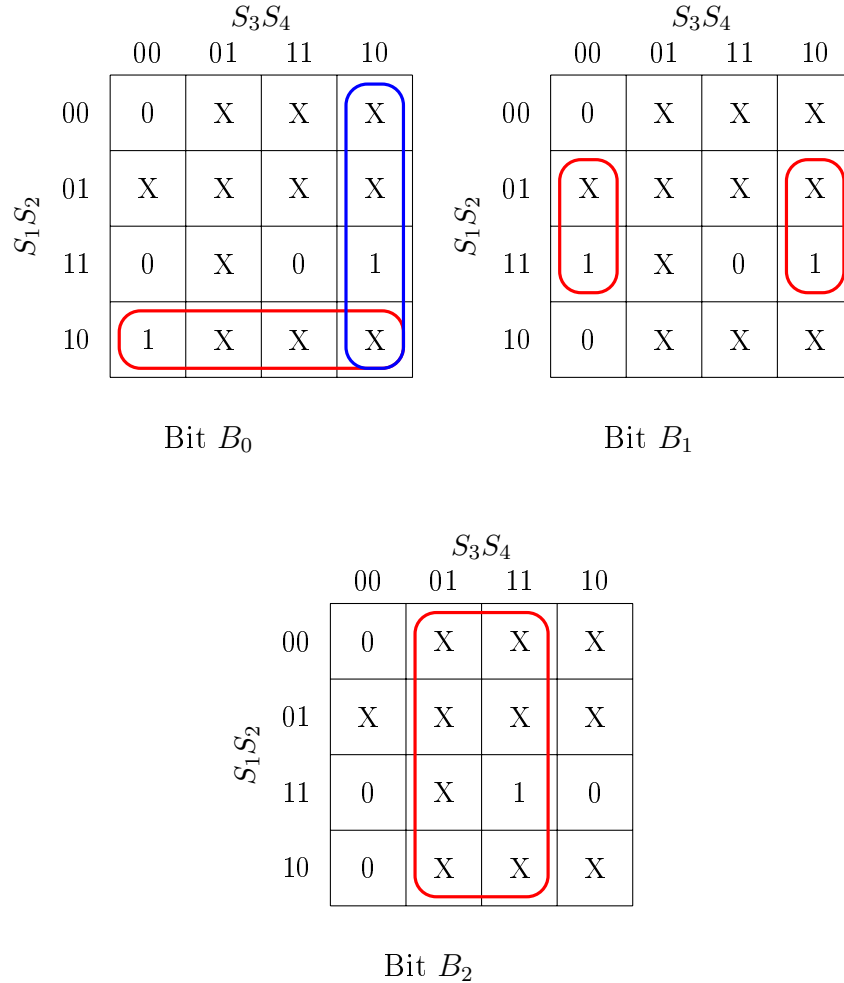


Figura 1: Mapas de Karnaugh de los tres bits de salida, con las señas inválidas marcadas como condiciones libres.

En el mapa de B_1 el grupo se dibuja partido en dos porque se envuelve por los bordes laterales del mapa, pero en realidad es un solo grupo de cuatro casillas, ya que las columnas de los extremos son adyacentes entre sí.

1.4 Ecuaciones obtenidas

De los tres mapas salieron las siguientes ecuaciones:

$$B_0 = S_1 \overline{S_2} + S_3 \overline{S_4}$$

$$B_1 = S_2 \overline{S_4}$$

$$B_2 = S_4$$

Hubieron dos resultados que nos parecieron interesantes. El primero es que B_2 se redujo a únicamente S_4 , sin necesitar ni una sola compuerta, y tiene todo el sentido del mundo si uno lo piensa, porque el único caso en que el valor llega a cuatro es cuando los cuatro dedos están arriba, entonces el sensor del último dedo ya carga esa información por sí solo.

El segundo es la forma que toma B_0 . Ese bit es el menos significativo, o sea que se enciende en los valores impares, que en nuestro caso son el 1 y el 3. El término $S_1 \overline{S_2}$ detecta exactamente la seña de un dedo, ya que es la única en la que el primer sensor está activo y el segundo no, y el término $S_3 \overline{S_4}$ detecta la de tres dedos por la misma razón. La suma de los dos casos nos da el bit completo.

1.5 Simulación en Proteus

Armamos el circuito dentro de un subcircuito jerárquico al que llamamos ENCODER, de manera que desde el esquemático principal solo se vean las cuatro entradas de los sensores y las tres salidas del valor binario. Trabajar por bloques nos mantiene el diseño ordenado y nos permite probar cada parte por separado antes de integrarla con el resto del cerrojo.

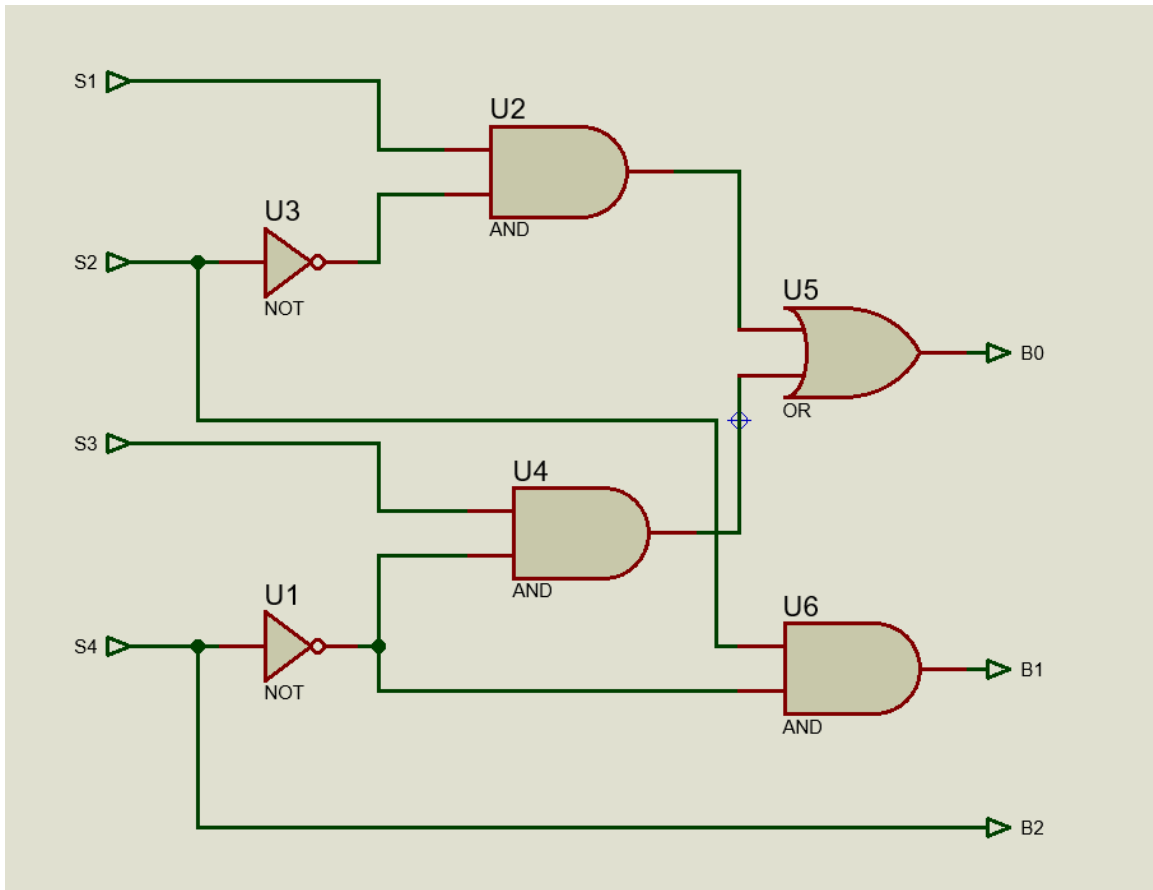


Figura 2: Implementación interna del codificador dentro del subcircuito ENCODER.

Para las pruebas conectamos generadores de estado lógico a cada uno de los sensores, lo que nos permite simular las señales manualmente sin tener que armar nada físico todavía.

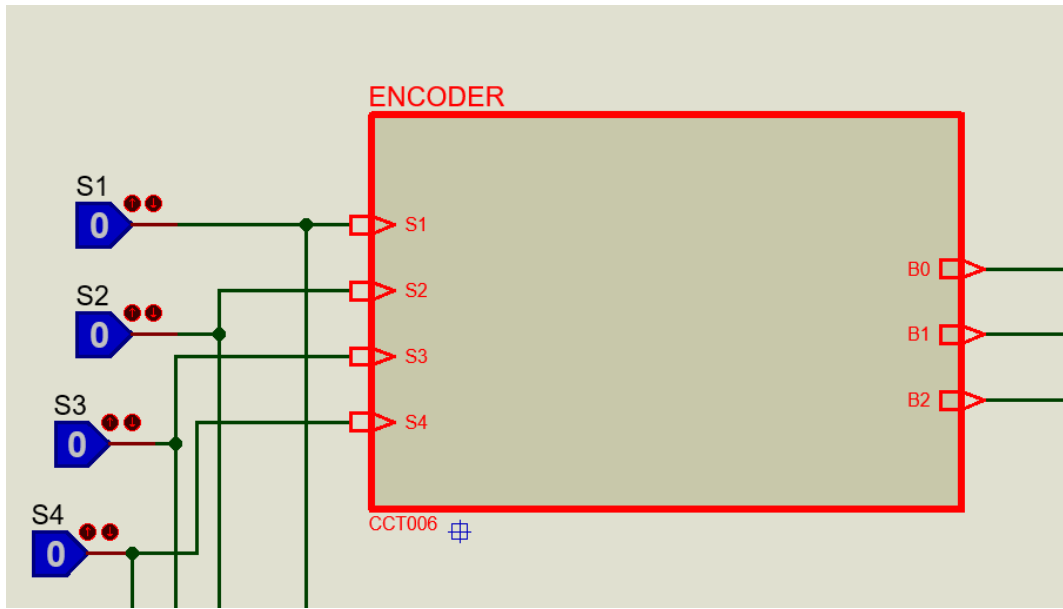


Figura 3: Bloque ENCODER visto desde el esquemático principal, con los generadores de estado lógico usados para las pruebas.

Recorrimos las cinco señas válidas una por una y en todas la salida coincidió con el valor esperado de la tabla de verdad, dado a que el circuito responde correctamente pasamos a integrarlo con el resto del diseño.

2 11 de septiembre de 2026

2.1 Verificador de señas

En la sesión anterior dejamos el codificador funcionando bajo el supuesto de que la seña que le llega ya respeta el orden de los dedos, por lo que el trabajo de hoy fue construir el bloque que se encarga de comprobar ese supuesto. El VERIFIER recibe las mismas cuatro entradas de los sensores que recibe el ENCODER y produce una única salida, a la cual llamamos INVALID, que se levanta cuando la combinación de dedos no corresponde a ninguna de las señas que el sistema acepta.

Separar esta responsabilidad en un bloque aparte nos permitió mantener el codificador mucho más limpio, limitando su trabajo únicamente a traducir la seña a binario y delegando la validación a un circuito propio. Además aunado a ello, la salida INVALID no se queda encerrada dentro del bloque, sino que más adelante la vamos a utilizar para bloquear el reloj, de manera que una seña mal formada no alcance a guardarse en el registro.

2.2 Tabla de verdad y ecuación del verificador

Empezamos escribiendo la tabla de verdad completa, con las 16 combinaciones que pueden formar los cuatro sensores, en donde marcamos cuáles corresponden a una seña válida y

cuáles no.

S_1	S_2	S_3	S_4	Válida	INVALID
0	0	0	0	1	0
0	0	0	1	0	1
0	0	1	0	0	1
0	0	1	1	0	1
0	1	0	0	0	1
0	1	0	1	0	1
0	1	1	0	0	1
0	1	1	1	0	1
1	0	0	0	1	0
1	0	0	1	0	1
1	0	1	0	0	1
1	0	1	1	0	1
1	1	0	0	1	0
1	1	0	1	0	1
1	1	1	0	1	0
1	1	1	1	1	0

Tabla 2: Tabla de verdad del verificador de señas.

Al revisar la tabla nos dimos cuenta de que las cinco filas válidas comparten una misma propiedad, la cual es que ningún dedo puede estar levantado si el dedo anterior no lo está también, y esta observación nos ahorró tener que armar un mapa de Karnaugh de cuatro variables, debido a que la condición se puede escribir directamente como tres reglas encadenadas: si S_2 está activo entonces S_1 tiene que estarlo, si S_3 está activo entonces S_2 tiene que estarlo, y si S_4 está activo entonces S_3 tiene que estarlo.

En álgebra booleana una implicación de la forma $A \rightarrow B$ equivale a $\overline{A} + B$, por lo que cada una de esas tres reglas nos da una suma de dos términos, y la seña es válida solamente cuando se cumplen las tres al mismo tiempo:

$$\text{VÁLIDA} = (S_1 + \overline{S_2})(S_2 + \overline{S_3})(S_3 + \overline{S_4})$$

Como lo que necesita el resto del circuito es la señal contraria, se aplicó De Morgan sobre esa expresión, con lo que obtuvimos la forma de suma de productos:

$$\text{INVALID} = \overline{S_1}S_2 + \overline{S_2}S_3 + \overline{S_3}S_4$$

Las dos ecuaciones son equivalentes y cuestan prácticamente lo mismo en compuertas, sin embargo se decidió a implementar la primera y negar el resultado al final, debido a que de esa forma cada compuerta del esquemático se corresponde con una de las tres reglas

que escribimos en palabras, lo que nos hace mucho más fácil revisar el circuito si en algún momento deja de funcionar. El costo de esa decisión es un inversor adicional a la salida, el cual nos pareció un precio más que razonable.

2.3 Implementación del verificador

El bloque quedó armado con nueve compuertas en total, las cuales son tres inversores para obtener los complementos de S_2 , S_3 y S_4 , tres compuertas OR para las tres sumas, dos compuertas AND para multiplicarlas entre sí y un último inversor para producir INVALID. El sensor S_1 es el único que no necesita inversor, dado a que en la ecuación aparece siempre sin negar.

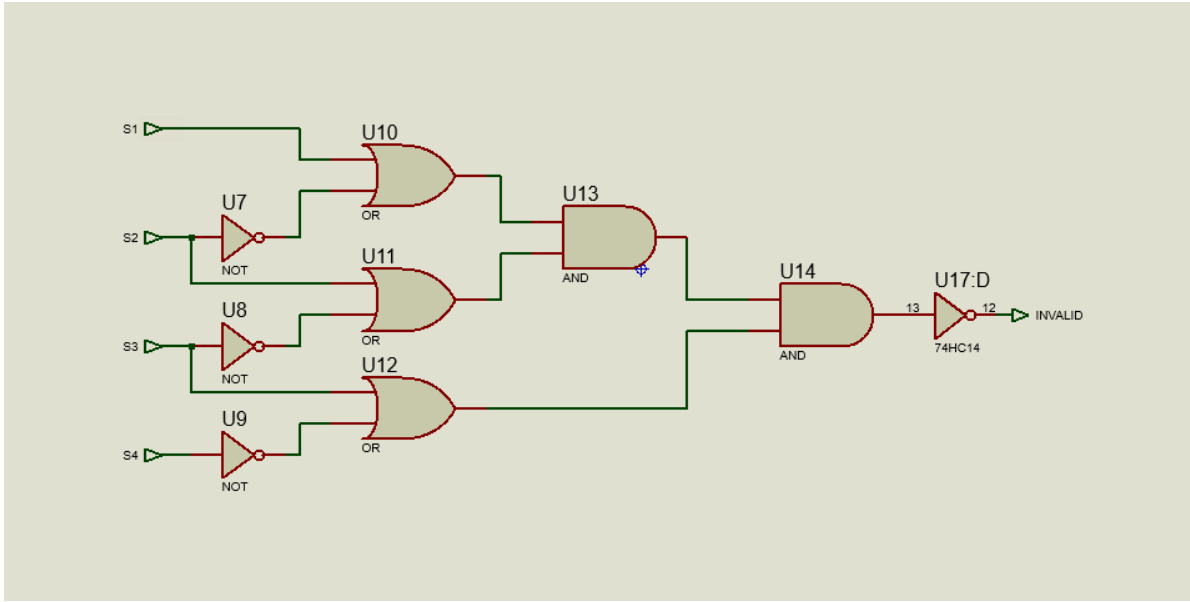


Figura 4: Implementación del verificador de señas dentro del subcircuito VERIFIER.

Probamos las 16 combinaciones una por una con generadores de estado lógico, en donde las cinco señas válidas mantuvieron la salida en cero, mientras que las otras once la levantaron tal como lo esperábamos.

2.4 Herramientas de apoyo

Consideramos importante dejar constancia de que a lo largo del proyecto hemos usado un asistente de inteligencia artificial (Claude, de Anthropic) como apoyo para estudiar y discutir el diseño. En el caso particular del verificador, la idea de describir la condición de orden como una cadena de implicaciones, en lugar de resolver un mapa de Karnaugh de cuatro variables, salió de una de esas conversaciones, al igual que el paso por De Morgan para obtener la forma de suma de productos. El armado del circuito en Proteus, la escogencia de las compuertas y las pruebas de las 16 combinaciones las hicimos nosotros.

2.5 Bloque de reloj

La segunda parte de la sesión fue el bloque de reloj, el cual es el que le da el ritmo a todo el cerrojo. De lo que se encarga es de producir exactamente tres pulsos cada vez que el usuario presiona el botón para confirmar un dígito, ni uno más ni uno menos, y de quedarse quieto hasta la siguiente pulsación, dado a que cada dígito de la contraseña son tres bits y el registro de desplazamiento los recibe de uno en uno.

2.6 Antirrebote del botón

Antes de poder contar pulsaciones tuvimos que limpiar la señal del botón, debido a que un botón mecánico no cambia de estado de una sola vez, sino que sus contactos rebotan durante unos cuantos milisegundos, y para un circuito digital cada uno de esos rebotes es una pulsación distinta, por lo que si dejáramos la señal cruda una sola pulsación del usuario le entraría al sistema como varias.

Se implementó un filtro pasa bajas seguido de un disparador de Schmitt. La resistencia R_3 de $10\text{ k}\Omega$ trabaja también como resistencia de bajada, la cual es la que mantiene la entrada en cero mientras el botón está sin presionar, y junto al capacitor C_2 de 100 nF forma una constante de tiempo de un milisegundo, que nos pareció más que suficiente para que los rebotes se promedien en lugar de pasar. El problema de este filtro es que a su salida la señal sube y baja de forma lenta, y una entrada lógica común no sabe qué hacer con un voltaje que se queda un rato en la zona intermedia, es por ello que la pasamos por dos inversores Schmitt del 74HC14, los cuales tienen umbrales distintos para subir y para bajar, lo que convierte la rampa en un flanco limpio. Utilizamos dos y no uno para que la señal salga con la misma polaridad con la que entró.

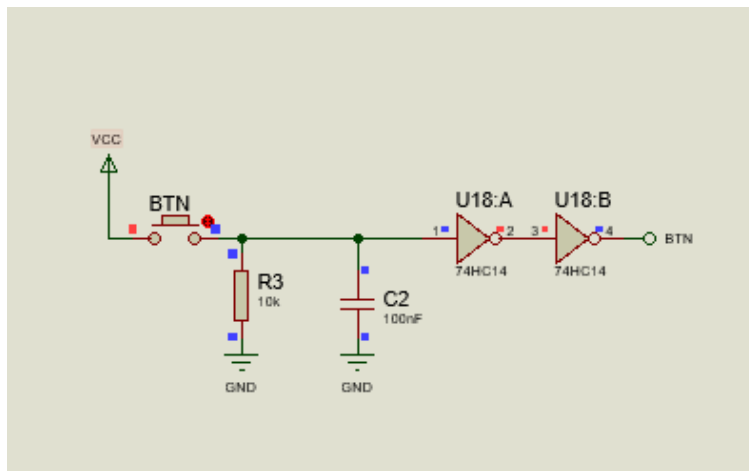


Figura 5: Circuito antirrebote del botón de confirmación.

2.7 Oscilador con el NE555

Para generar los pulsos utilizamos un NE555 en configuración astable, la cual es la que hace que el integrado oscile por sí solo, sin necesidad de que nadie lo dispare. La frecuencia

depende de las dos resistencias y del capacitor de tiempo, según la siguiente relación:

$$f = \frac{1.44}{(R_1 + 2R_2) C_1}$$

Con $R_1 = 1 \text{ k}\Omega$, $R_2 = 10 \text{ k}\Omega$ y $C_1 = 100 \text{ nF}$ nos queda una frecuencia de aproximadamente 686 Hz, es decir un período de 1.46 ms, lo que significa que los tres pulsos que necesitamos para capturar un dígito se completan en unos 4.4 ms, que es un tiempo muchísimo más corto que el que una persona tarda en soltar un botón, por lo que el dígito siempre alcanza a entrar completo dentro de una misma pulsación.

El capacitor C_3 de 10 nF que va al pin de control no participa en el cálculo de la frecuencia, de lo que se encarga es de aterrizar el voltaje de referencia interno del integrado para que el ruido de la fuente no le mueva los umbrales de comparación, y la resistencia R_4 de 300 Ω a la salida limita la corriente que el 555 le entrega a la entrada lógica que sigue.

2.8 Contador de pulsos y señal DONE

El 555 por sí solo oscila todo el tiempo, por lo que hacía falta algo que lo dejara pasar únicamente durante los tres pulsos que nos interesan, lo cual resolvimos con un cerrojo SR armado con dos compuertas NOR realimentadas entre sí, en donde la entrada de encendido es la señal del botón ya limpia y la de apagado es una señal a la cual llamamos DONE. Cuando el usuario presiona, el cerrojo se activa y su salida habilita una compuerta AND que deja pasar la oscilación del 555 hacia el resto del sistema, y cuando llega DONE el cerrojo se apaga y la AND vuelve a bloquear el reloj.

DONE la produce un contador 74HC393, del cual utilizamos una de sus dos mitades. El contador recibe el reloj ya filtrado y su entrada de reinicio va conectada a la señal del botón invertida, lo que hace que mientras el botón esté suelto el contador se mantenga en cero y que empiece a contar apenas se presiona. Con una compuerta AND entre sus salidas Q_0 y Q_1 detectamos el valor tres, el cual es justo la cantidad de pulsos que queremos, y esa detección es la que apaga el cerrojo.

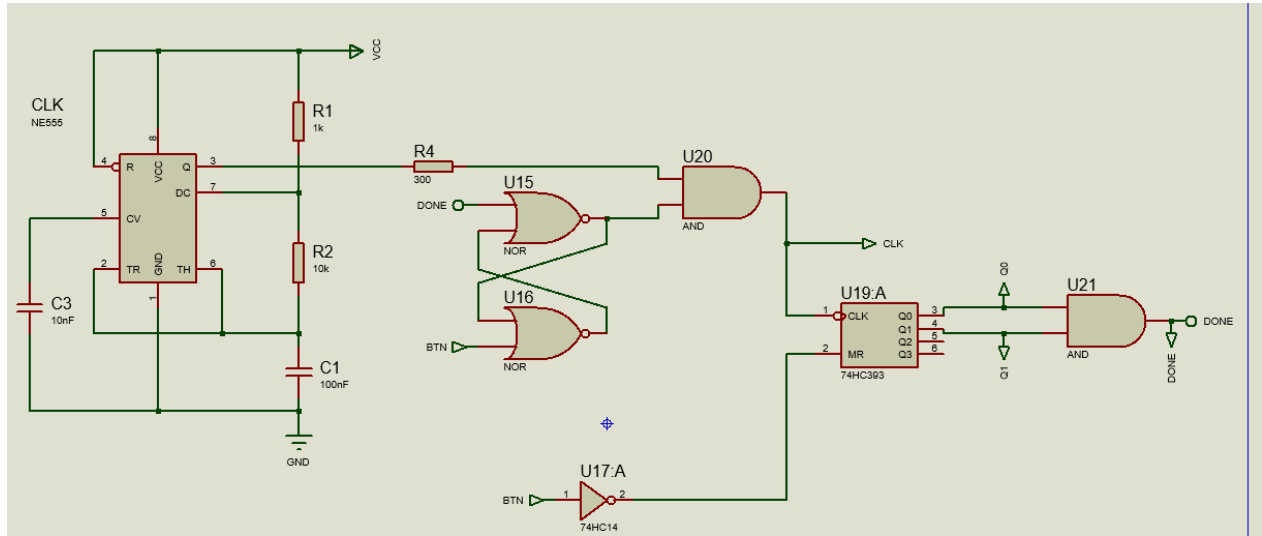


Figura 6: Bloque de reloj completo, con el oscilador, el cerrojo de habilitación y el contador que genera la señal DONE.

Un detalle que consideramos importante anotar es que DONE no solo sirve para frenar el reloj, ya que cuando el usuario suelta el botón el contador se reinicia y DONE se cae, y ese flanco de bajada es una señal perfectamente limpia que ocurre una única vez por cada dígito ingresado, la cual más adelante vamos a aprovechar como reloj del contador de dígitos, que es el que lleva la cuenta de cuántos números de la contraseña van entrados.

Simulamos el bloque completo y verificamos con el osciloscopio que por cada pulsación del botón salen tres flancos y nada más, sin importar cuánto tiempo se mantenga presionado, es por ello que consideramos el bloque terminado y de una vez pasamos a conectarlo con el serializador.

2.9 Serializador

El serializador es el bloque que conecta al ENCODER con la memoria del cerrojo, y de lo que se encarga es de tomar los tres bits que el codificador entrega al mismo tiempo y pasárselos al registro de desplazamiento de uno en uno, dado a que este último no los puede recibir de ninguna otra forma.

Para esto utilizamos un multiplexor 74HC4052, al cual le conectamos B_0 , B_1 y B_2 en los canales X0, X1 y X2, y del cual obtenemos un único bit por el pin X, que es el que alimenta la entrada de datos del primer registro. Las entradas A y B del integrado son las que deciden cuál de los canales se deja pasar, por lo que necesitábamos algo que nos las fuera cambiando en cada pulso de reloj.

En primera instancia pensábamos armar un contador aparte para manejar esas dos entradas, pero después nos dimos cuenta de que el contador que ya habíamos hecho en el bloque de reloj nos servía igual, debido a que mientras cuenta los tres pulsos sus salidas Q_0 y Q_1 van tomando los valores 0, 1 y 2, que son justamente los números de canal que necesitamos. Es

por ello que se decidió a reutilizarlo, lo que nos permitió resolver la selección sin agregarle ni una compuerta adicional al circuito.

El cuarto canal (X3) no lo utiliza el proyecto, sin embargo lo conectamos a tierra en lugar de dejarlo al aire, y lo mismo hicimos con el banco Y completo, el cual es la otra mitad del multiplexor que tampoco ocupamos. La entrada INH también va a tierra, dado a que es la que habilita el integrado y la necesitamos activa todo el tiempo.

2.10 Filtrado del reloj

En este bloque fue en donde conectamos por fin la salida del verificador. La señal que les llega a los registros no es el reloj crudo que produce el NE555, sino una versión filtrada a la que llamamos FILTERED_CLK, la cual se corta mientras la seña que está haciendo el usuario no sea válida, lo que impide que una combinación mal formada alcance a guardarse en la memoria.

Se implementó con un inversor y una compuerta NOR, cuyo comportamiento queda descrito por la siguiente ecuación:

$$\text{FILTERED_CLK} = \overline{\overline{\text{CLK}} + \text{INVALID}} = \text{CLK} \cdot \overline{\text{INVALID}}$$

Es decir que el pulso pasa únicamente cuando hay reloj y además la seña es válida. Se utilizó una NOR y no una AND seguida de un inversor, debido a que de está forma nos ahorramos una compuerta, dado a que el inversor de la entrada lo íbamos a necesitar de todas maneras.

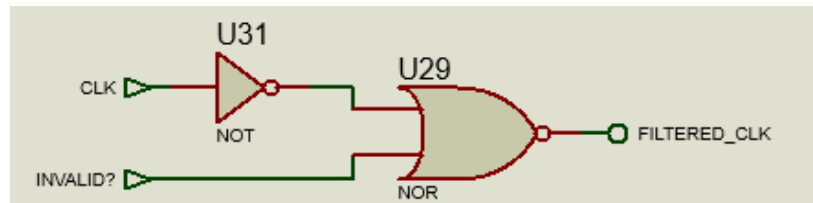


Figura 7: Filtrado del reloj con la señal INVALID del verificador.

Hay un detalle de sincronización que consideramos importante dejar anotado. El contador 74HC393 avanza con los flancos de bajada del reloj, mientras que los registros CD4015 guardan el bit con los flancos de subida, por lo que primero sube el reloj y el registro captura el bit que el multiplexor tiene seleccionado en ese momento, y solo después baja el reloj y el contador pasa al siguiente canal. Las dos cosas nunca ocurren al mismo tiempo, lo que le da al multiplexor tiempo de sobra para estabilizarse antes de que le pidan el dato.

2.11 Cadena de registros de desplazamiento

La contraseña son nueve bits, los cuales salen de tres dígitos de tres bits cada uno. Para guardarlos utilizamos registros CD4015, quiénes traen dos registros de cuatro etapas dentro

de un mismo integrado, entonces con tres mitades llegamos a doce etapas, de las cuales ocupamos nueve.

La cadena se arma conectando la última salida de un registro con la entrada de datos del siguiente. Un detalle del CD4015 que hay que tener presente a la hora de cablearlo es que esa última salida no sigue la secuencia de los pines, ya que en una mitad las salidas van por los pines 13, 12, 11 y la cuarta salta al pin 2, mientras que en la otra van por el 5, 4, 3 y la cuarta salta al pin 10. Esos dos pines son precisamente los que llevan el encadenamiento, es por ello que los dejamos anotados desde ahora para no equivocarnos cuando armemos el circuito en físico.

Las entradas de reinicio de los tres registros las conectamos a tierra. En primera instancia las íbamos a conectar a la misma señal de reinicio que usa el contador, pero al analizarlo nos dimos cuenta de que no hacía falta, debido a que un registro de desplazamiento se limpia solo, ya que después de nueve desplazamientos cualquier bit viejo ya salió de las nueve etapas que nos interesan, por lo que lo único que queda guardado son los nueve bits que el usuario acaba de ingresar. Esto funciona porque el reloj filtrado y el contador van amarrados entre sí, es decir que siempre son exactamente tres desplazamientos por cada dígito.

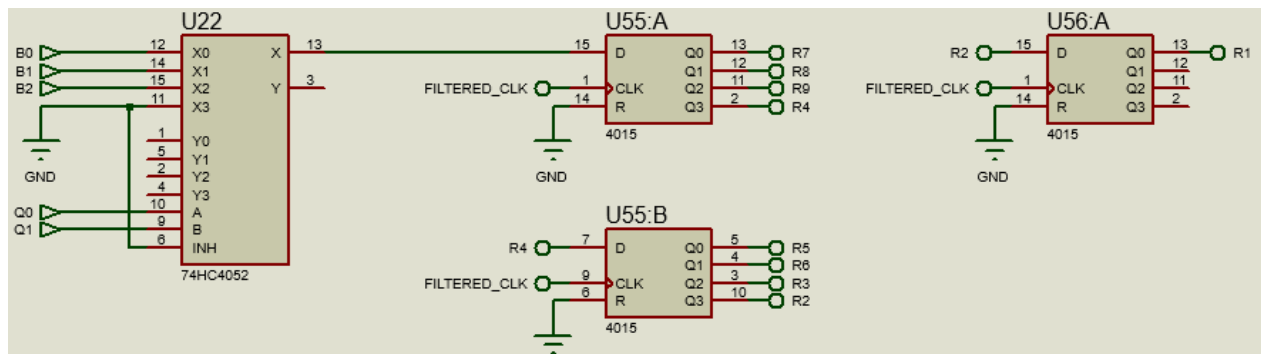


Figura 8: Serializador completo, con el multiplexor y la cadena de tres registros de desplazamiento.

2.12 Orden de los bits en el registro

La parte que más nos costó de todo el bloque fue determinar en qué orden quedan guardados los bits. En un registro de desplazamiento el primer bit que entra es el que más lejos viaja, dado a que los que vienen atrás lo van empujando, por lo que cuando terminan los nueve desplazamientos el bit del primer dígito queda en la última etapa y el del último dígito queda en la primera, es decir que la contraseña se guarda al revés de como se ingresó.

Para no confundirnos armamos la siguiente tabla, la cual es la referencia que vamos a utilizar de aquí en adelante para saber qué contiene cada salida. La numeración de las etiquetas no sigue un patrón evidente, debido a que salió de acomodarlas al orden en que las necesita el comparador, es por ello que la tabla es la única fuente confiable.

Etapa	Salida	Etiqueta	Contenido	Valor en 2-4-1
1	U55:A Q_0 (13)	R_7	Dígito 3, bit B_2	0
2	U55:A Q_1 (12)	R_8	Dígito 3, bit B_1	0
3	U55:A Q_2 (11)	R_9	Dígito 3, bit B_0	1
4	U55:A Q_3 (2)	R_4	Dígito 2, bit B_2	1
5	U55:B Q_0 (5)	R_5	Dígito 2, bit B_1	0
6	U55:B Q_1 (4)	R_6	Dígito 2, bit B_0	0
7	U55:B Q_2 (3)	R_3	Dígito 1, bit B_2	0
8	U55:B Q_3 (10)	R_2	Dígito 1, bit B_1	1
9	U56:A Q_0 (13)	R_1	Dígito 1, bit B_0	0

Tabla 3: Contenido de cada etapa del registro después de ingresar tres dígitos, con los valores que corresponden a la contraseña 2-4-1.

La última columna la calculamos a mano para poder comprobar el bloque, en donde el dígito 2 en binario es 010, el 4 es 100 y el 1 es 001. Ingresando esa secuencia en la simulación las nueve salidas tenían que quedar exactamente como dice la tabla y así fue, con lo cual quedamos seguros de que el orden que dedujimos es el correcto y ya podemos utilizar estas etiquetas para armar el comparador de la contraseña.

2.13 Comparador de contraseñas

El comparador es el bloque encargado de revisar si los nueve bits que el serializador guardó en los registros corresponden a la contraseña correcta. Para este proyecto, el cerrojo tiene dos contraseñas distintas dependiendo de lo que el usuario quiera hacer: para abrir la puerta es 2-4-1, y para cerrarla es 2-4-3.

En primera instancia pensábamos armar dos comparadores por aparte y elegir cuál usar dependiendo del estado de la puerta, pero al analizar los bits de las dos contraseñas nos dimos cuenta de que son casi idénticos. Si revisamos los valores, el dígito 1 (2) y el dígito 2 (4) son iguales en ambas. En el caso del dígito 3, el 1 es binario 001 y el 3 es 011. La única diferencia entre las dos contraseñas es el bit B_1 del tercer dígito, el cual según la tabla que armamos corresponde a la etapa conectada a R_8 .

Para abrir la puerta (cuando la señal OPEN? está en 0), necesitamos que la contraseña sea 2-4-1, por lo que R_8 debe ser 0. Para cerrar la puerta (cuando OPEN? está en 1), necesitamos que sea 2-4-3, es decir que R_8 debe ser 1. Es por ello que se decidió a utilizar una única compuerta XNOR 4077, la cual compara R_8 directamente con la señal OPEN?. Dado a que la compuerta XNOR produce un 1 lógico solo cuando ambas entradas son iguales, esta nos resuelve la validación dual sin necesidad de duplicar el circuito completo.

El resto de las salidas del registro (R_1 a R_7) se pasan por inversores 74HC14 en los casos donde el bit esperado es un 0, esto para que todos los caminos entreguen un 1 lógico cuando la contraseña ingresada es la correcta. Posteriormente, se multiplican todas estas señales, junto a la salida de la XNOR y a R_9 (que es 1 en ambas contraseñas y pasa directo), utilizando un árbol de compuertas AND.

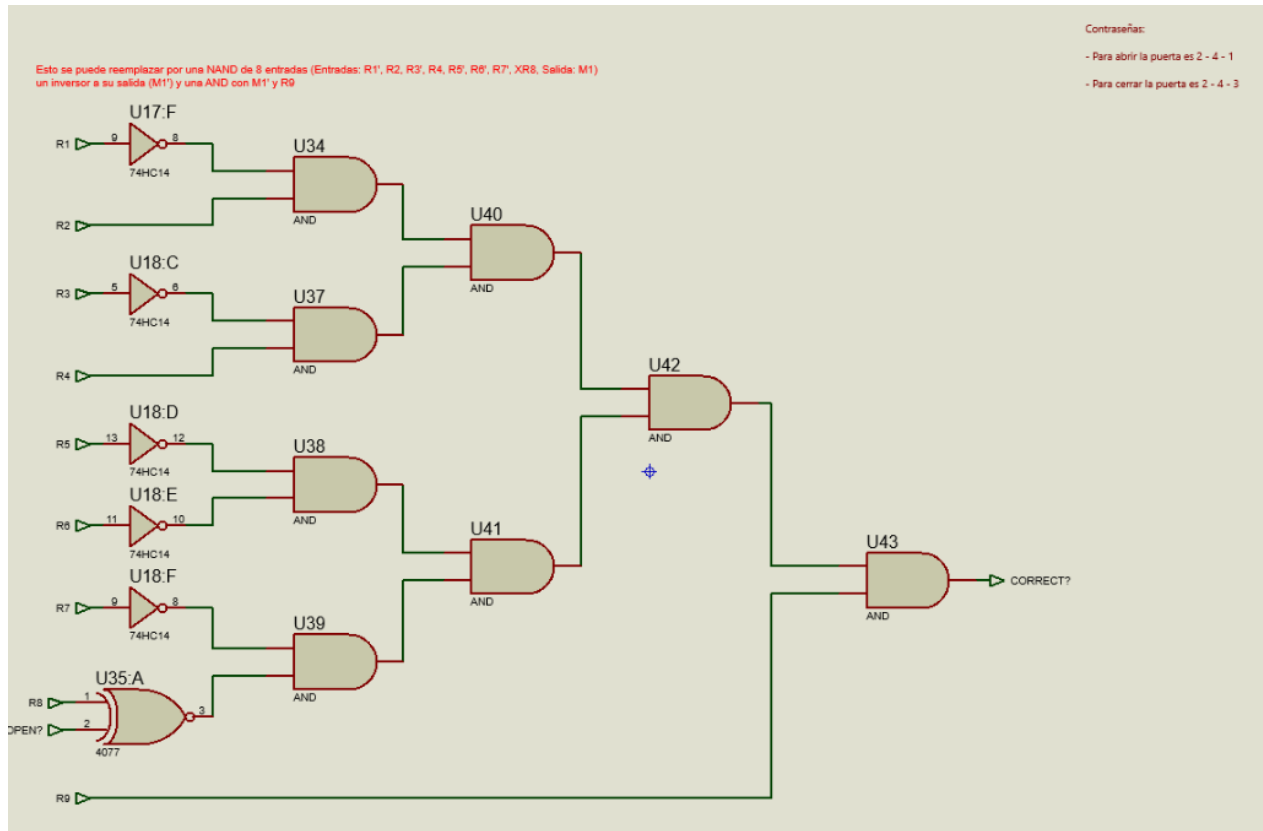


Figura 9: Interior del subcircuito PSW_CHECKER, con la compuerta XNOR para el bit R_8 y el árbol de compuertas AND.

Cabe destacar que el árbol de compuertas AND resulta ser muy grande y ocupa bastantes integrados. Como observación para el momento de armar el circuito en físico, nos dimos cuenta de que esto se puede reemplazar por una compuerta NAND de 8 entradas, a la cual le conectamos las señales desde R_1 hasta R_8 (pasadas por la lógica mencionada), seguida de un inversor y una última compuerta AND con la señal de R_9 . Esto nos ahorraría una cantidad importante de espacio en las protoboards, además aunado a ello reduce la cantidad de cables.

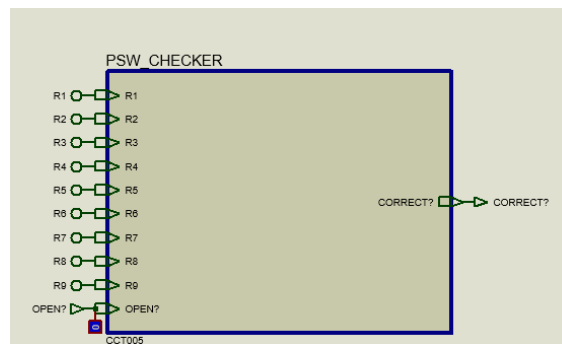


Figura 10: Bloque PSW_CHECKER visto desde el exterior, con las entradas de los registros y la señal OPEN?.

Con el comparador terminado, la señal CORRECT? ya está lista para avisarle al resto del sistema si la seña ingresada sirve o no para mover el motor.

3 13 de septiembre de 2026

3.1 Controlador de estados

El STATE CONTROLLER es el bloque encargado de decidir qué hace el cerrojo. Recibe DONE del reloj, CORRECT? del comparador y las dos señales de los finales de carrera, y con eso determinará en cuál de cuatro situaciones se encuentra el aparato, las cuales quedan codificadas en dos bits que llamamos E_1 y E_0 .

COMPLETE?	CORRECT?	OPENED?	E_1	E_0	Situación
0	X	X	0	0	Leyendo
1	0	X	0	1	Seña inválida
1	1	0	1	0	Abriendo
1	1	1	1	1	Cerrando

Tabla 4: Estados del cerrojo y su codificación en dos bits.

Este bloque necesita dos datos que ninguno de sus vecinos le entrega, que son la cantidad de dígitos ingresados y la posición de la puerta, es por ello que adentro del controlador hay dos circuitos con memoria que trabajan antes de la lógica de estados.

3.2 Contador de dígitos

El reloj entrega DONE cada vez que termina de capturar un dígito, entonces contando esos pulsos el bloque sabrá cuántos van. Para esto utilizamos la mitad libre del 74HC393, dado a que ese integrado trae dos contadores adentro y en el bloque de reloj solo ocupamos uno.

Una compuerta AND entre las salidas S_0 y S_1 detecta el valor tres y produce la señal COMPLETE?, lo que permite avisarle al resto del bloque que ya hay una contraseña completa esperando a ser evaluada.

3.3 Cerrojo de posición de la puerta

Mientras la puerta va en camino ninguno de los dos finales de carrera está presionado, por lo que el sistema necesita recordar en dónde quedó.

Armamos un cerrojo SR con dos compuertas NOR realimentadas entre sí, en donde SW_OPEN lo enciende y SW_CLOSED lo apaga. Su salida OPENED? se mantendrá fija en el último valor mientras ninguno de los dos interruptores esté presionado.

3.4 Monoestable de reinicio

En primera instancia el reinicio del contador se iba a conectar directamente a los dos finales de carrera, o sea $MR = SW_OPEN + SW_CLOSED$, pero al analizarlo nos dimos cuenta de que la puerta se queda descansando encima de uno de los dos interruptores, por lo que esa señal quedaría levantada de forma permanente y el contador no podría contar nunca.

Es por ello que se decidió a convertir ese nivel en un pulso corto, utilizando un monoestable armado con un capacitor y un disparador de Schmitt. El capacitor C_4 va en serie y la resistencia R_7 a tierra, que es al revés del antirrebote, lo que permite que el circuito deje pasar únicamente el instante del cambio.

Con R_7 de 10 k Ω y C_4 de 100 nF el pulso queda de aproximadamente un milisegundo. La resistencia R_8 de 1 k Ω que sigue al capacitor protege la entrada del integrado y no participa en ese cálculo.

El monoestable se disparará en tres situaciones, las cuales se juntan con compuertas OR antes del capacitor: cuando la puerta llega al extremo abierto, cuando llega al cerrado, y cuando el usuario presiona el botón teniendo una contraseña completa ya evaluada.

Está última condición se agregó después de encontrar un error en la simulación, en donde una contraseña equivocada dejaba el contador en tres y en la siguiente pulsación el conteo seguía subiendo en lugar de reiniciarse, con lo que el contador terminaba desfasado del registro de desplazamiento. Con el tercer disparo el contador se limpia apenas el usuario vuelve a presionar.

3.5 Ecuaciones de estado

Con COMPLETE?, CORRECT? y OPENED? ya disponibles, los dos bits salen de estas dos expresiones:

$$\begin{aligned}E_1 &= COMPLETE? \cdot CORRECT? \\E_0 &= COMPLETE? \cdot (OPENED? + \overline{CORRECT?})\end{aligned}$$

Las dos llevan COMPLETE? multiplicando, lo que permite que mientras no haya una contraseña completa el estado se mantenga en 00, que es el de lectura. Cuando sí la hay, E_1 se levanta únicamente si la contraseña resultó correcta, entonces ese bit significa por sí solo que la puerta se moverá.

E_0 se levanta en dos casos, ya sea porque la contraseña estuvo mala, que da el estado 01, o porque estuvo buena pero la puerta ya estaba abierta, que da el 11 y corresponde a cerrarla.

3.6 Salidas hacia el motor

$$\begin{aligned}\text{OPENING} &= E_1 \cdot \overline{OPENED?} \\ \text{CLOSING} &= E_1 \cdot OPENED?\end{aligned}$$

Las dos salen de E_1 y no de la combinación completa de los dos bits, debido a que E_1 ya significa que la puerta se moverá y lo único que falta decidir es hacia dónde, cosa que responde OPENED?.

Además aunado a ello, escritas de está forma quedan mutuamente excluyentes por construcción, lo cual es importante dado a que estas dos señales manejarán el puente H del motor.

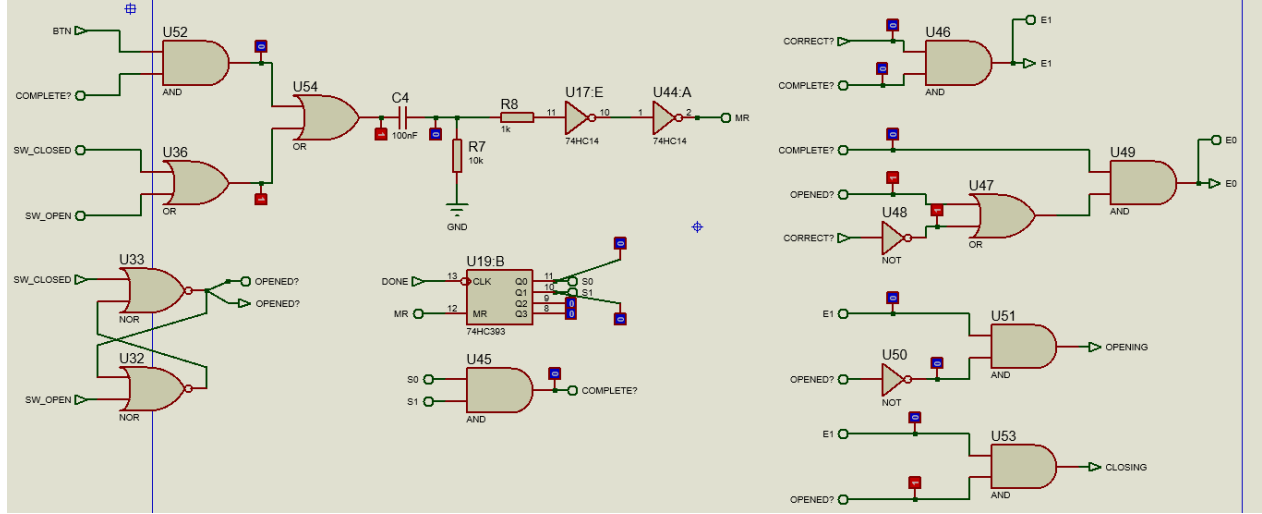


Figura 11: Interior del STATE CONTROLLER, con el monoestable de reinicio, el cerrojo de posición, el contador de dígitos y la lógica de estados.

3.7 Decodificador de anuncios

El decodificador de anuncios traduce los dos bits del estado a las siete señales del display, lo que permite que el usuario sepa qué está pasando en cada momento.

E_1E_0	Letra	a	b	c	d	e	f	g
00	L	0	0	0	1	1	1	0
01	E	1	0	0	1	1	1	1
10	A	1	1	1	0	1	1	1
11	C	1	0	0	1	1	1	0

Tabla 5: Segmentos que se encienden en cada una de las cuatro letras.

En primera instancia los siete segmentos se iban a resolver con puras compuertas a partir de E_1 y E_0 , pero al final se decidió a meter antes un decodificador 74HC238, el cual toma los dos bits y levanta una sola de sus salidas Y_0 a Y_3 según el estado.

Sus dos habilitaciones activas en bajo van a tierra, la activa en alto va a VCC, y la entrada C también a tierra porque solo ocupamos cuatro de sus ocho salidas. Trabajar sobre esas cuatro señales en lugar de sobre los dos bits nos permitió dejar las ecuaciones mucho más fáciles de leer, ya que cada segmento se convierte en la lista de los estados en los que tiene que encenderse:

$$a = \overline{Y_0}$$

$$d = Y_0 + Y_1 + Y_3$$

$$e = f = 1$$

$$b = c = Y_2$$

$$g = \overline{Y_0} Y_1 + Y_2$$

Los segmentos e y f están encendidos en las cuatro letras, entonces no necesitan ni una compuerta y se conectan directo a VCC con su resistencia, dado a que las letras L, E, A y C, quiénes son las únicas que el display mostrará, comparten el lado izquierdo completo.

Revisando el circuito ya armado encontramos que g se puede reducir, debido a que las salidas del decodificador son mutuamente excluyentes y el término $\overline{Y_0}$ no aporta nada, quedando la expresión en $Y_1 + Y_2$. Aplicarlo libera un inversor y una compuerta AND, y se hará antes del armado físico.

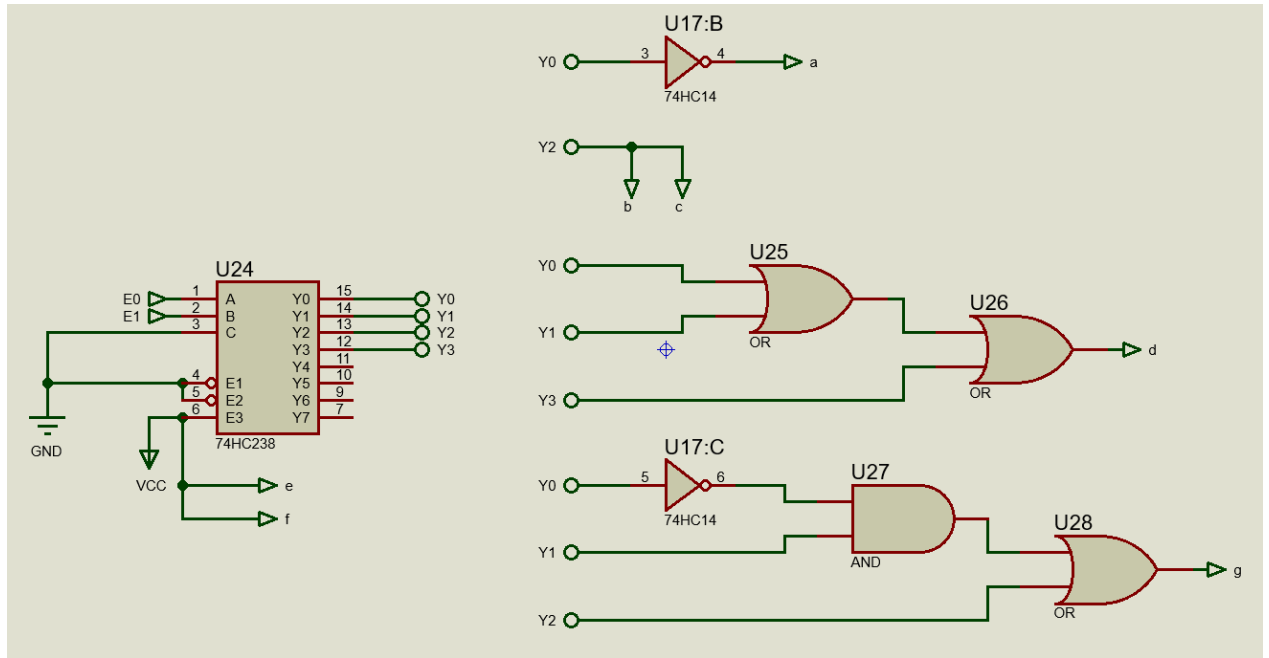


Figura 12: Interior del decodificador de anuncios, con el 74HC238 y la lógica de los siete segmentos.

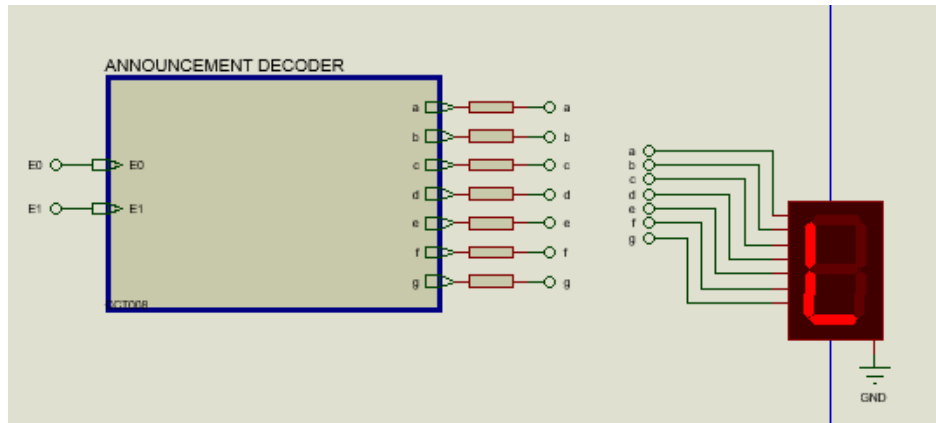


Figura 13: Bloque del decodificador conectado al display de siete segmentos.

3.8 Simulación

Recorrimos los cuatro estados uno por uno en la simulación y en todos el display mostró la letra que le corresponde. También comprobamos que OPENING y CLOSING nunca se levantan al mismo tiempo.

Después de una contraseña equivocada el sistema vuelve a quedar listo apenas se presiona el botón, que era el comportamiento que fallaba antes de agregarle el tercer disparo al monoestable.

Con esto el diseño digital del cerrojo queda completo, lo que permite empezar a repartir el trabajo para armarlo en físico.

4 Repositorio de GitHub

El código fuente, los esquemáticos de Proteus y demás recursos de este proyecto se encuentran alojados en el siguiente repositorio:

<https://github.com/Mandamientos/Cerrojo-CE1107>

Referencias Bibliográficas

- [1] Texas Instruments. (2014). *NE555 Precision Timers*. Datasheet.
- [2] Texas Instruments. (2004). *SN74HC14 Hex Schmitt-Trigger Inverters*. Datasheet.
- [3] Texas Instruments. (2003). *CD4015B CMOS Dual 4-Stage Static Shift Register*. Datasheet.
- [4] Labcenter Electronics. *Proteus Design Suite*. Software de automatización de diseño electrónico y simulación de circuitos.